作业 1：反向传播与神经网络

深度学习（84100343-0）　清华大学

2026 年秋季　中文译本

引言

构建一个由专家组成的小型语言模型。莎士比亚文本和 Python 代码遵循不同的模式，但都可以通过预测下一个符号来建模。是否每个 token 都必须使用同一个前馈网络? 你将构建一个混合专家（MoE）模型，并研究它如何在控制每个 token 计算量的同时提升模型容量。

MoE 层存储若干个 MLP，称为*专家*。一个经过学习的*路由器*会为每个 token 选择少数几个专家，并组合它们的输出，从而将总容量与实际参与的计算分开。专家通过联合训练学习各自的作用，而非人工指定。Switch Transformers 在大规模场景探索了这一原理 [1]。Mixtral 8x7B 在语言、代码和多语言任务中为每个 token 从八个专家中选择两个 [2]。DeepSeek-V3 报告总计 6710 亿参数、每个 token 激活 370 亿参数 [3]。本作业将在课程作业规模下研究这一机制。

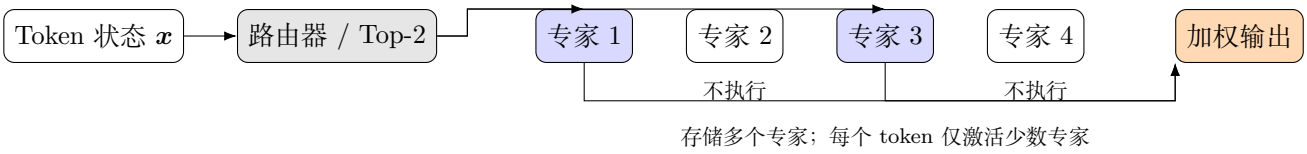


图 1: 一个 token 激活所存储专家中的一个小子集。参照 Mixtral [2] 图 1 的路由模式原创绘制的示意图；此处为便于说明仅画出四个专家。

我们的下一字节预测器包含三个步骤：推导基本模块、用 NumPy 验证它们，以及测试一个小型因果语言模型。专家采用常见的全连接（FC）层和 ReLU；因果上下文模块已提供。请遵守以下要求：

- 在书面报告（PDF）中逐项作答，语言可为英文或中文，并提供清晰且带注释的代码。不得分享或提交他人的作业。
- **[手写]** 题要求独立手写推导。请将作业拍照或扫描后插入 PDF。不得使用 AI 生成推导；可用计算器进行数值近似。
- 未标记的题目要求自行编写核心代码；AI 可以解释一般语法或环境错误，但不能提供该核心代码。[AI 探索] 题鼓励借助 AI 探索。请披露所获协助及讨论内容；不得捏造结果。

总分：100 分（手写 35 分，核心编程 50 分，探索 15 分）。代码和数据随本 PDF 提供；请在发布包目录下运行命令。

深度学习（2026 年秋季），清华大学。

1 第一部分：反向传播（35 分）

1.1 引言

稀疏 MoE 层为每个 token 表示选择少数几个前馈专家 [1,2]。其组成部分都很常见：全连接层、ReLU、Softmax、加权求和和残差连接。我们先研究一个小型可微示例。局部卷积提供上下文表示；第三部分会用已提供的因果上下文模块替换此编码器。

1.2 网络细节

给定输入 $\boldsymbol{s} \in \mathbb{R}^4$，共享卷积核 $\boldsymbol{q} \in \mathbb{R}^3$ 产生

$$a_j = \sum_{t=1}^{3} q_t s_{j+t-1}, \qquad h_j = \mathrm{ReLU}(a_j), \qquad j = 1, 2. \tag{1}$$

不使用填充，步幅为 1，运算为互相关。该编码器给出 $D = 2$；下游模块按一般 D 编写。取 $\mathrm{ReLU}'(0) = 0$。对于 E 个专家、隐藏宽度 H 和 K 个预测类别，定义

$$\boldsymbol{v}_e = W_{1,e}\boldsymbol{h} + \boldsymbol{b}_{1,e}, \qquad \boldsymbol{u}_e = \mathrm{ReLU}(\boldsymbol{v}_e), \qquad \boldsymbol{f}_e = W_{2,e}\boldsymbol{u}_e + \boldsymbol{b}_{2,e}, \tag{2}$$

$$\boldsymbol{g} = \mathrm{Softmax}(G\boldsymbol{h}), \qquad \boldsymbol{r} = \boldsymbol{h} + \sum_{e=1}^{E} g_e \boldsymbol{f}_e, \qquad \hat{\boldsymbol{y}} = \mathrm{Softmax}(C\boldsymbol{r} + \boldsymbol{b}_c). \tag{3}$$

其中 $W_{1,e} \in \mathbb{R}^{H \times D}$、$W_{2,e} \in \mathbb{R}^{D \times H}$、$G \in \mathbb{R}^{E \times D}$、$C \in \mathbb{R}^{K \times D}$。所有偏置的维度与其对应输出相同；路由器没有偏置。对大小为 m 的小批量，使用

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \log \hat{y}_k^{(i)}. \tag{4}$$

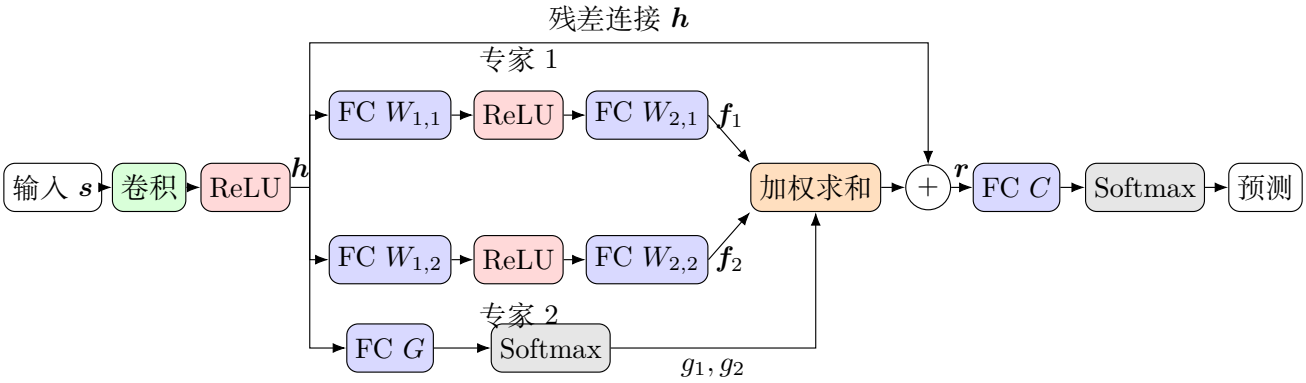


图 2: 手写推导示例网络。蓝、红、灰、橙色框分别表示全连接层、ReLU、Softmax 和加权求和；圆形表示加法。两个专家均被选中。

数值计算使用 $m = 1, D = H = E = 2, K = 3$，参数如下。未列出的专家偏置均为零：

$$\boldsymbol{s} = (2, 1, -1, 3)^\top, \qquad \boldsymbol{q} = (1, \tfrac{1}{2}, \tfrac{1}{2})^\top, \qquad \boldsymbol{y} = (0, 1, 0)^\top,$$

$$W_{1,1} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad W_{1,2} = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}, \quad \boldsymbol{b}_{1,1} = \begin{pmatrix} 0 \\ 1/2 \end{pmatrix}, \quad \boldsymbol{b}_{1,2} = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

$$W_{2,1} = \begin{pmatrix} 1/5 & 0 \\ 0 & 1/5 \end{pmatrix}, \quad W_{2,2} = \begin{pmatrix} 0 & 1/5 \\ 1/5 & 0 \end{pmatrix}, \quad \boldsymbol{b}_{2,1} = \begin{pmatrix} 0 \\ 1/2 \end{pmatrix}, \quad \boldsymbol{b}_{2,2} = \begin{pmatrix} 1/2 \\ 0 \end{pmatrix},$$

$$G = \begin{pmatrix} 1/4 & -1/4 \\ 0 & 0 \end{pmatrix}, \quad C = \begin{pmatrix} 1/5 & 0 \\ 0 & 1/5 \\ 1/5 & 1/5 \end{pmatrix}, \quad \boldsymbol{b}_c = (-0.49, -0.49, -0.98)^\top. \tag{5}$$

1.3 题目

请展示必要的推导和变量维度。强烈建议使用向量形式；中间伴随量可以重复使用。

1. **[手写]** 计算式 (5) 对应的 $\boldsymbol{h}$、两个专家的输出、$\boldsymbol{g}$、$\boldsymbol{r}$、预测值和损失。(6 分)
2. **[手写]** 推导 Softmax 雅可比矩阵，以及交叉熵对预测 logits 的梯度。说明小批量平均如何进入梯度。(6 分)
3. **[手写]** 对一般的 m, D, H, E, K，推导 $W_{1,e}, \boldsymbol{b}_{1,e}, W_{2,e}, \boldsymbol{b}_{2,e}, G, C, \boldsymbol{b}_c$ 以及每个输入 $\boldsymbol{h}$ 的梯度。明确包含专家、路由器和残差路径。仅数值计算 $\partial\mathcal{L}/\partial G_{1,1}$。(16 分)
4. **[手写]** 通过共享卷积反向传播。可使用经核验的上游梯度 $\nabla_{\boldsymbol{h}}\mathcal{L} = (349, -173)^\top/2400$。计算 $\nabla_{\boldsymbol{q}}\mathcal{L}$、$\partial\mathcal{L}/\partial s_2$，并给出一次学习率为 0.1 的卷积核更新。为所有可训练参数给出小批量 SGD 伪代码。(7 分)

2 第二部分：实现专家（20 分）

2.1 引言

使用 NumPy 实现式 (2)–(4) 中的模块。不允许使用自动微分。完成 `NumPy/model.py`；数据、初始化、中心差分检查器和训练循环均已提供。批量样本按行存储；参数数组遵循上文的列向量约定。

2.2 稀疏路由与梯度验证

对于选出的 k 个专家，令 $S(\boldsymbol{h})$ 表示路由 logits 中最大的 k 个所对应的索引，并将式 (3) 中的门控替换为

$$g_e = \begin{cases} \dfrac{\exp((G\boldsymbol{h})_e)}{\sum_{j \in S(\boldsymbol{h})} \exp((G\boldsymbol{h})_j)}, & e \in S(\boldsymbol{h}), \\ 0, & e \notin S(\boldsymbol{h}). \end{cases} \tag{6}$$

选中集合在局部固定。检查器会提供远离 Top-k 并列情况和 ReLU 折点的掩码；无需对离散选择求导。检查器使用 $\epsilon = 10^{-6}$，并报告最大归一化误差

$$\max_j \frac{|a_j - n_j|}{\max(1, |a_j| + |n_j|)}, \qquad n_j = \frac{\mathcal{L}(\theta + \epsilon e_j) - \mathcal{L}(\theta - \epsilon e_j)}{2\epsilon}. \tag{7}$$

2.3 题目

1. 实现前向与反向传播，包括预测头和共享输入梯度。复现第一部分，并支持所提供的三专家 Top-2 情形。（10 分）
2. 运行 `python NumPy/run.py --check`。报告每类参数和输入的误差，目标是误差低于 10^{-5}。诊断所提供的错误输入梯度变体，并指出缺少了哪条计算路径。（6 分）
3. 在所提供的同一语料库的 128 个样本批次上运行 `python NumPy/run.py --train`。使用 400 次 SGD 更新和 0.2 的学习率；绘制损失曲线并展示三个预测。上下文特征固定。不要求进行超参数搜索或达到泛化目标。（4 分）

3 第三部分：小型 MoE 语言模型（45 分）

3.1 数据集

在 Tiny Shakespeare [4] 与 CPython 源代码 [5] 的固定混合数据上预测下一个字节。词表由全部 256 种可能字节组成。软件包提供了分词后的数据和固定划分：Shakespeare 使用连续的 80/10/10 百分比划分；代码按文件划分。窗口不会跨越文件或数据划分边界。训练时两个数据域等权采样。域标签仅用于评估，绝不作为路由器输入。

3.2 网络细节

所提供的双块因果模型维度为 $D = 64$，有四个注意力头，上下文长度为 $T = 64$。因果上下文模块只使用当前及之前的字节。嵌入、归一化、注意力、残差包装器、优化器和负载均衡均已提供；不评估其内部实现。只需完成 `MoE/model.py` 中标记的核心方法。本部分允许使用 PyTorch 自动微分。

每个专家使用 ReLU，将维度 $64 \to 128 \to 64$。一个块的结构是 $\boldsymbol{x} + \mathrm{MoE}(\mathrm{LN}(\boldsymbol{x}))$，并采用式 (6)，其中 $E = 16, k = 2$。第一部分为手算省略了归一化包装器。这是用于教学的小规模 token 级 FFN 路由模型，并非对预训练大型模型的复现。

表 1: 三种配置；其余模型组件均相同。参数量匹配为近似值（误差在 1% 以内）。

配置	专家数	选中数	FFN 隐藏宽度
计算量匹配的稠密模型	1	1	264
稀疏 MoE	16	2	每个专家 128
参数量匹配的稠密模型	1	1	2056

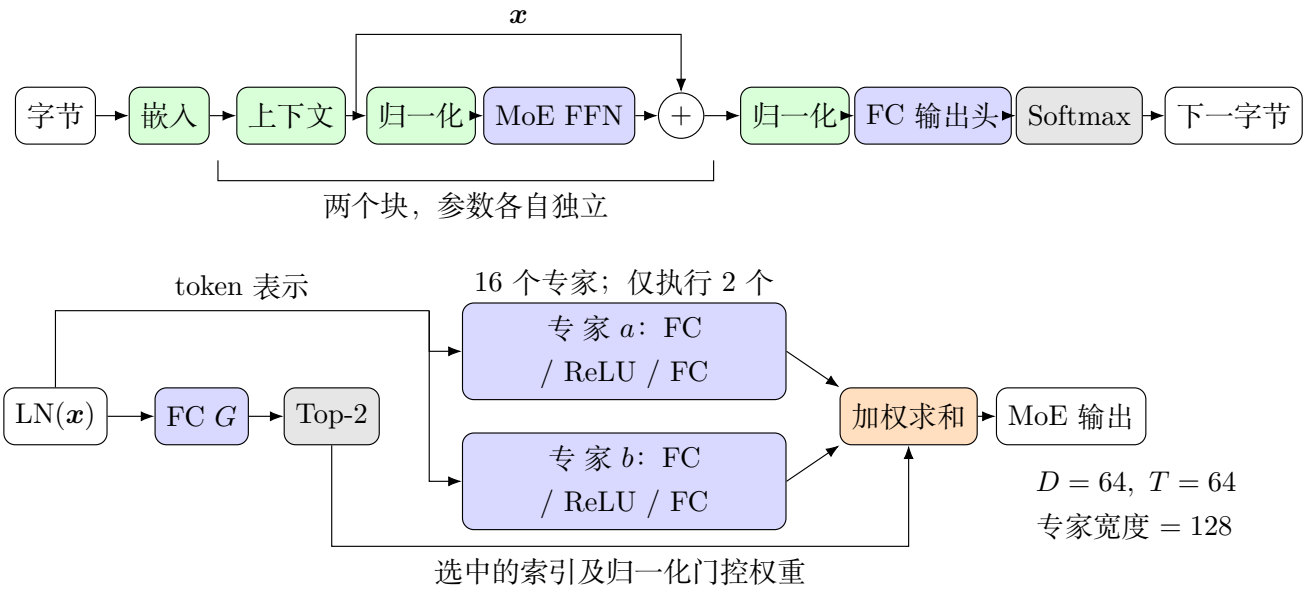


图 3: 训练模型。两个块结构相同，但参数独立。仅被选中的 token–专家对执行专家 MLP。绿色为已提供模块，蓝色为全连接层，灰色为门控或概率。

3.3 训练与评估

使用随机种子 416、3000 次更新，以及由 32 个长度为 64 的序列组成的批次。保留提供的优化器、学习率计划、数据顺序和检查点规则：选取验证交叉熵最低的检查点。辅助负载均衡项的系数为 0.01，无需手工推导。评估时使用留出的、互不重叠且长度完整的数据块。分别报告两个数据域，并以

$$\mathcal{L}_{\text{test}} = \tfrac{1}{2}(\mathcal{L}_{\text{text}} + \mathcal{L}_{\text{code}}), \qquad \text{PPL} = \exp(\mathcal{L}_{\text{test}}) \tag{8}$$

作为主要指标。只有在词元化方式和评估协议相同的前提下，字节级困惑度才可比较。本比较固定更新次数和数据顺序，不固定墙钟时间或收敛程度。评分取决于正确性和分析，而非 MoE 是否胜出或是否达到绝对速度/PPL 目标；不同设备间可能存在小的数值差异。

3.4 题目

1. 实现专家前向计算、路由器和所选 token 的分发。先运行提供的 CPU 检查 `python MoE/diagnostics.py --check`；然后执行 `python MoE/train.py --mode all`。报告全部三种配置的训练/验证曲线和测试指标。不要先计算所有专家再屏蔽其输出。(10 分)

2. 在同一设备上比较质量、总参数量、每 token 的矩阵乘加（MAC）次数，以及完整批次前向延迟 ($B = 32, T = 64$)。MAC 统计要包括注意力、路由器、专家和输出头；计时要包括分发开销。运行 `python MoE/diagnostics.py`：在最后一层同一位置上，不同序列之间交换成对路由，同时保持专家–token 数量不变。该程序会检查前缀不变性，并报告三个固定打乱种子下的诊断交叉熵 CE。这种依赖批次的扰动不代表语言模型独立运行时的性能；不要据此推断因果训练收益。用一张表和一幅图讨论容量、计算、路由和专家使用情况。仅凭使用量不能证明语义专门化；较低 MAC

也不一定意味着更低延迟。(20 分)

3. [**AI 探索**] 设计一个小型自主研究。选择一个关于稀疏计算、路由或模型行为且可检验的问题。AI 工具可协助构思、编程和制图。(15 分)

可选的起始方向（也欢迎其他问题）：(a) 在一个 Top-2 检查点上比较推理时 $k = 1, 2, 4$：质量与 MAC 的关系；(b) 固定最后一层选中的专家，比较学习得到的门控权重与相等门控权重（各为 1/2）：仅仅选对专家是否足够?

明确提出问题和有意义的对照；围绕必做实验结果，通过少量实验或分析收集证据。解释你的理解及一个局限；不要求结果有所改进。研究报告最多 600 个英文单词或 1000 个中文字，图表最多两幅；请提供可运行代码和 AI 使用声明。评分依据问题（3 分）、设计（4 分）、证据（4 分）、解释（3 分）和披露（1 分）。

4 提交格式

提交名为 `{StudentID}_{Name}_hw1.zip` 的压缩包，其中包含：

1. 一份逐题作答的 PDF，包括第一部分的手写照片、梯度检查结果、实验比较和探索研究。
2. 第二、三部分可运行的核心代码、配置和精简结果日志。请包含必要的参考文献和协助披露；无需包含完整模型检查点和所提供的数据集。

AI 使用声明

参照 ICLR 的做法，请在报告结尾用 3–5 句话披露所用工具/模型、受影响的题目/任务及核验方式，或明确声明未使用 AI。确认遵守以上规则，并对提交内容负责。本声明不计入 600 词/1000 字的限制。

参考文献

[1] W. Fedus, B. Zoph, and N. Shazeer. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. JMLR, 23(120), 2022.

[2] A. Q. Jiang et al. Mixtral of Experts. arXiv:2401.04088, 2024.

[3] DeepSeek-AI et al. DeepSeek-V3 Technical Report. arXiv:2412.19437, 2025.

[4] A. Karpathy. Tiny Shakespeare corpus, char-rnn. https://github.com/karpathy/char-rnn.

[5] Python Software Foundation. CPython 3.10.16 source distribution. https://www.python.org/downloads/release/python-31016/.